

# CV

## Miloš Vasić

**Инженер — LLM-инфраструктура, автономные агенты и системы управления, обеспечивающие их надёжность.**

- Email: milos85vasic@gmail.com
  - Веб: <https://milosvasic.ru> · <https://vasic.digital>
  - GitHub: vasic-digital · HelixDevelopment · Server-Factory
- 

### Краткое описание

Инженер-программист, создающий AI-системы от начала до конца — от мультипровайдерской LLM-инфраструктуры и автономных агентов до слоёв контроля качества и управления, которые гарантируют их прозрачность. Я не создаю демоверсии — я разрабатываю платформы. За 15 лет профессиональной работы (с 2009 года) в области мобильных SDK, интеграции с реальным оборудованием и распределённых бэкендов мой опыт теперь сосредоточен на одной цели: сделать автономную AI-разработку надёжной в промышленных масштабах.

Я проектирую флоты, а не монолиты — крупные приложения, состоящие из десятков небольших, развязанных и независимо протестированных модулей, каждый из которых наследует общие инженерные Constitution и проверяется по принципу антиблефа с доказательной системой контроля качества. Основной язык — Go, также использую Kotlin/KMP, TypeScript/React, Python, Swift и Shell. Главный принцип, реализуемый не на словах, а технически: функция считается готовой только тогда, когда её может использовать реальный пользователь и есть зафиксированные доказательства её работоспособности.

**Что я привношу в проект:** умение превратить AI-технология из исследовательской идеи в управляемую, самопроверяющуюся систему промышленного уровня — LLM-маршрутизацию, которая доказывает работоспособность каждой модели до её внедрения, агентов, способных спорить и

достигать консенсуса вместо догадок, слой памяти и RAG, не теряющие контекст, и целую экосистему, где «тесты прошли» никогда не будет означать «функция сломана».

## Ключевые компетенции

- **AI / LLM-системы:** абстракция мультипровайдерской LLM-инфраструктуры (40+ провайдеров), интеграция MCP-инструментов, RAG, vector-базы данных и embeddings, оркестрация агентов (безголовые CLI-агенты, графовые рабочие процессы, многораундовые дебаты/консенсус), планирование (HiPlan/MCTS/Tree-of-Thoughts), LLMOps, бенчмаркинг (SWE-bench/HumanEval/MMLU), верификация LLM, защитные механизмы LLM, компьютерное зрение + LLM-зрение.
- **Бэкенд-разработка:** Go (Gin), gRPC + Protobuf, HTTP/3 (QUIC), WebSockets, распределённые системы (включая формальные спецификации TLA+), высокопроизводительные REST-сервисы и конкурентные воркеры.
- **Работа с данными:** PostgreSQL, SQLite, SQLCipher (шифрование на уровне хранения), Redis, Neo4j, ClickHouse, объектные хранилища (MinIO/S3/GCS/Azure).
- **Фронтенд / кроссплатформенная разработка:** TypeScript/React (Tailwind, Redux Toolkit, i18next), Angular, Electron, React Native, Kotlin Multiplatform, Android/Android TV (Kotlin), iOS (Swift), Tauri/Rust.
- **Инфраструктура / DevOps:** Docker и Compose, Kubernetes + Helm, Prometheus + Grafana, OpenTelemetry, QEMU/Libvirt/Parallels; CI/CD через GitHub Actions, Gradle, Make.
- **Контроль качества / инженерия качества:** доказательный контроль качества по принципу антиблефа (HelixQA), тестовые стенды с мутационными проверками, go test -race, визуальное регрессионное тестирование, тестирование устройств через ADB, SonarQube, сканирование безопасности (semgrep/gosec/trivy/snyk/gitleaks/nancy).
- **Управление инженерными процессами:** Constitution как подмодуль; механизмы наследования и распространения; дисциплина документирования и покрытия тестами в флоте из 140+ репозиториев.

## Избранные проекты

### Управление и контроль качества

- **HelixConstitution** — универсальный свод инженерных правил, распространяемый как Git-сабмодуль и

наследуемый более чем в 140 репозиториях: инженерные нормы поставляются и фиксируются по версиям точно так же, как код. Одно обновление сабмодуля обновляет правила для всей инфраструктуры, а механизмы распространения буквально проверяют каждый зависимый репозиторий на наличие требуемой оговорки — каждый из них сопровождается метатестом на мутации, доказывающим, что сам механизм не фиктивный. Это превращает «лучшие практики, которым люди надеются следовать» в наследуемый, поддающийся аудиту и механически соблюдаемый закон без права на обман.

- **HelixQA** — оркестрация контроля качества без права на обман (Go), построенная на одном непреклонном правиле: планка не в том, что «тесты проходят», а в том, что «пользователи могут пользоваться функцией». Она запускает написанные банки тестов YAML и полностью автономные сессии QA на базе LLM и компьютерного зрения, которые открывают реальное приложение, проверяют каждую задокументированную функцию, выискивают недокументированные баги на Android/Android TV/Web/Desktop и отказываются засчитывать результат как УСПЕХ без подтверждающих данных времени исполнения — скриншотов, логов logcat, видео, стектрейсов — а также готовых к исправлению тикетов AI.

## Разработка AI и инфраструктура LLM

- **HelixAgent** — промышленный ансамблевый сервис LLM (Go/Gin), который отказывается доверять одной модели: он распределяет запрос между множеством провайдеров, проводит структурированные многораундовые дебаты (Предложение → Критика → Обзор → Синтез) и маршрутизирует запросы по результатам проверки в реальном времени с плавным откатом — всё это за совместимым с OpenAI интерфейсом API с высокодоступным слоем данных, наблюдаемостью и защитными механизмами. *Go, Gin, PostgreSQL, Redis, Prometheus/Grafana/OpenTelemetry, MCP, Neo4j/ClickHouse/Kafka.*
- **HelixCode** — распределённая платформа разработки AI, которая делит работу на интеллектуальные задачи с учётом зависимостей в управляемом SSH парке воркеров, а затем сохраняет контрольные точки и откатывается, чтобы ничто не терялось при прерывании задачи; выбор модели с учётом аппаратных возможностей и полный цикл планирования/сборки/тестирования/рефакторинга за интерфейсами REST/CLI/TUI/MCP. *Go, Gin, PostgreSQL, Redis, SSH, MCP, llama.cpp/Ollama.*
- **HelixLLM** — один бинарный файл, шесть режимов развёртывания: совместимый с OpenAI и Anthropic инференс поверх HTTP/3, масштабируемый от ноутбука до

многохостового кластера с локальным инференсом через llama.cpp (CUDA/Metal/ROCm) и автоматически обнаруживаемой цепочкой отката в облако с проверкой результатов, которая всегда деградирует до гарантированно работающей локальной модели. *Go, HTTP/3 QUIC, gRPC/SSE/Kafka, llama.cpp*.

- **LLMProvider / LLMOrchestrator / LLMsVerifier** — основа инфраструктуры LLM: единый интерфейс для 43 провайдеров с автоматическими выключателями, мониторингом работоспособности, повторными попытками с экспоненциальным откатом и честным (без жёстко закодированных запасных вариантов) обнаружением моделей; потокобезопасная контрольная плоскость, которая запускает и управляет безголовыми агентами CLI (OpenCode, Claude Code, Gemini, Junie, Qwen Code) через гибридный протокол pipe+файл; а также эталонный источник проверки, чей обязательный гейт «Видишь мой код?» означает, что пометаться как пригодные или экспортироваться могут только те модели, которые действительно работают.
- **HelixMemory / HelixSpecifier** — унифицированный движок когнитивной памяти, объединяющий четыре лучших в своём классе бэкенда (Mem0, Cogneе, Letta, Graphiti) за одним интерфейсом с параллельным поиском и переранжированием результатов из разных источников; а также движок слияния спецификаций, управляемый разработкой на основе спецификаций, который масштабирует собственную процедуру под объём работы и подкрепляет спецификации многоагентными дебатами.
- **HelixTrack** — альтернатива JIRA + Confluence для свободного мира (флагман линейки Helix-Track): микросервисы Go с унифицированным интерфейсом API с маршрутизацией действий поверх HTTP/3, шифрованием данных в состоянии покоя SQLCipher и нативными клиентами для Web/Desktop/Android/iOS.

## Инструменты производственного уровня (vasic-digital utils)

- **Catalogizer** — мультипротокольный, зашифрованный, саморазвёртываемый менеджер медиакolleкций (Go/Gin + React), устойчивый к нестабильным сетевым хранилищам, построенный на базе 21 переиспользуемого субмодуля.
- **Courses-Creator** — конвейер преобразования Markdown в видео AI с поддержкой TTS и плеерами для настольных, мобильных и веб-платформ.
- **VisionEngine** — компьютерное зрение + мультипровайдерский интерфейс восприятия LLM с навигационными графами.
- **DocProcessor · Docs Chain · Herald · task\_bridge · Набор переиспользуемых модулей Vasic Digital** —

картографирование функций для QA, синхронизация документов и баз данных с хешированием контента, уведомления на естественном языке, синхронизация задач и досок, а также флот стандартной библиотеки `digital.vasic.*`.

## Автоматизация инфраструктуры (Server Factory)

- **Mail Server Factory** — декларативное развёртывание JSON в полностью подготовленные Docker-контейнеры почтовых серверов для 12 типов подключений и 25 дистрибутивов Linux; прошло 439 тестов и успешно преодолело контрольную точку SonarQube.
- **Базовый фреймворк Server Factory, Qemu-Utills, Parallels-Utills** — общий движок развёртывания и инструментарий для работы с образами виртуальных машин.

## Языки и инструменты (краткий список)

Go · Kotlin · Kotlin Multiplatform · TypeScript · JavaScript · Python · Swift · Java · Rust · Shell · PL/pgSQL · TLA+ · Gin · gRPC · HTTP/3 · React · Angular · Electron · React Native · PostgreSQL · SQLite · SQLCipher · Redis · Neo4j · ClickHouse · Docker · Kubernetes · Prometheus · Grafana · OpenTelemetry · QEMU · GitHub Actions · Gradle · Make

## Опыт работы

*Разработчик программного обеспечения с 2009 года, опыт работы на всех этапах жизненного цикла разработки — от планирования и программирования до руководства командой и внедрения. Полная история ниже основана на проверенных данных кандидата ([milosvasic.ru](http://milosvasic.ru)).*

## Полная занятость

- **Разработчик SDK — Harness** ([harness.io](http://harness.io)), Белград, Сербия · 03/2020 – 12/2024. Ведущий разработчик семейства SDK для подразделения Feature Flag компании, с фокусом на все основные мобильные платформы и не только. Клиенты и партнёры: AWS, Google и различные банки. Технологии: Android, iOS, Flutter, React Native, TypeScript, JavaScript, Java, Kotlin, Swift, Go, Ruby.
- **Инженер-программист — Leica Geosystems** ([leica-geosystems.com](http://leica-geosystems.com)), Хербругг, Швейцария · 02/2016 – 02/2020.

Разработка ПО для iOS и Android для передовых 3D-сканеров Leica Geosystems — работа с оборудованием в реальном времени, обработка данных и синхронизация. Партнёр: Autodesk. *Технологии: Android, iOS, Java, Kotlin, Swift, C++.*

- **Разработчик SDK — Bosch** (bosch.rs), Белград, Сербия · 01/2010 – 01/2016. Ведущий разработчик SDK для проекта Connected Vehicles — обмен данными Bluetooth с шиной OBD2 в реальном времени, высокопроизводительная обработка и хранение данных. *Технологии: Android, Java, Kotlin.*

## Прочие проекты

- **TN-TECH** (tn-tech.co.rs), Нови-Сад, Сербия · частичная занятость, с 03/2017. Работа в Globex Data (Канада и Швейцария) — Sekur (SekurMessenger), SekurMail, SekurSuite — и платформа BusRide. *Технологии: Android, Java, Kotlin, C++, Qt.*
- **Increment Loop** (incrementloop.com), Белград, Сербия · частичная занятость, с 09/2023. Приложение Yuno. *Технологии: Android, Kotlin.*
- **Открытые проекты / собственные инициативы** — HelixTrack, Server Factory (Mail Server Factory, Parallels-Utills, Qemu-Utills), а также Vasic Digital (Android-Toolkit, Network-Binder), подробнее см. в разделе «Избранные проекты» выше.

## Публикации

- **«Основы Kotlin»** — авторское издание; последнее переработанное издание — сентябрь 2022 г. («Основы Kotlin», 3-е издание). Также публиковался в издательстве Packt Publishing (Великобритания).

## Образование

- **Магистр современных информационных технологий** — Университет Singidunum, Белград, Сербия · 2014 г.
- **Бакалавр информатики и вычислительной техники** — Университет Singidunum, Белград, Сербия · 2008 г.